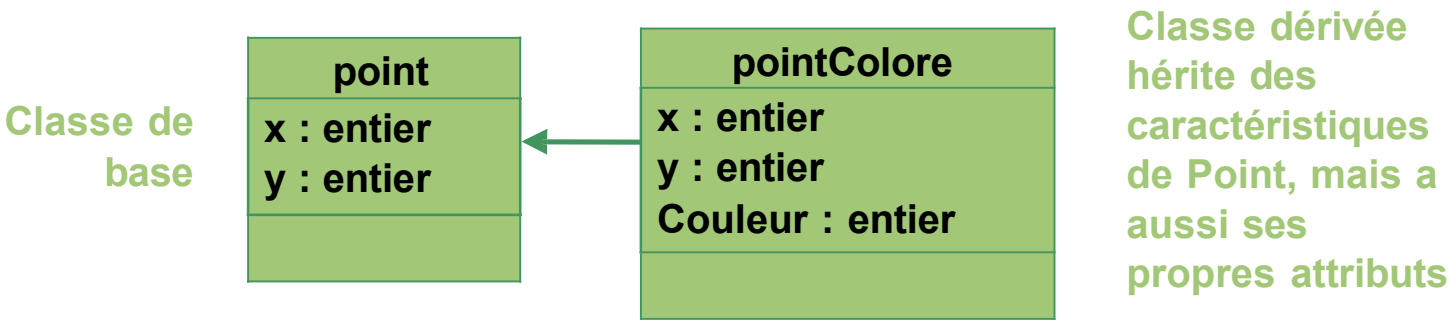
LANGAGE PYTHON

```python
#!/usr/bin/env python3
# -*- Coding: utf-8 -*-
"""

Created on Wed May 11 13:35:35 2022

@author: soukeina
"""


import numpy as np
import pandas as pd

#"""
# classe Series
tableau=np.random.randint(21, size=20)
serie_pd=pd.Series(tableau)
print(serie_pd)
comptage=serie_pd.value_counts()
print(comptage)
print("Valeur(s) la(es) plus fréquente(s), fréquence :")
print(comptage[comptage==max(comptage)])

#"""

# classe DataFrame
mydataset = {'cars': ["BMW", "Volvo", "Ford"], 'passagers': [3, 7, 2]}
mds_panda = pd.DataFrame(mydataset)
print(mds_panda,"\n")
print("Voiture à la plus grande capacité :")
print(mds_panda[mds_panda['passagers']==max(mds_panda['passagers'])])
```

PRINCIPE DE L'HÉRITAGE

- Créer facilement de nouvelles classes à partir de classes existantes
- Exemple :



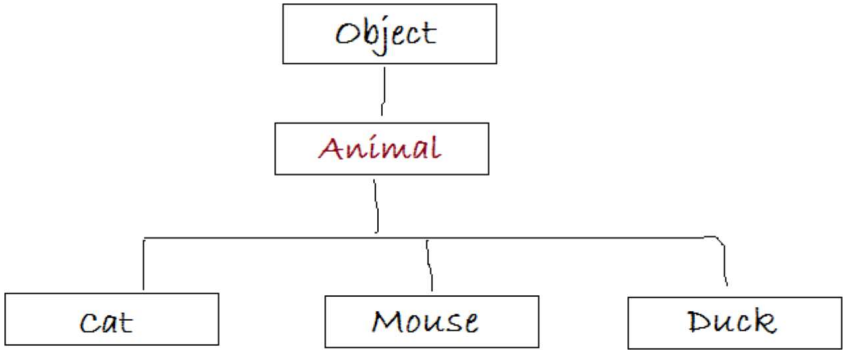
- La classe Point est une généralisation de la classe PointColore : un point coloré est un point, avec une autre spécificité.

L'HÉRITAGE EN PYTHON

- Python vous permet de créer une classe étendue d'une simple classe ou de multiples classes.
- Cette classe est appelée une classe dérivée, ou simplement une sous classe.
- La sous-classe hérite des attributs, des méthodes et des autres membres de la classe mère.
- Elle peut également redéfinir les méthodes de la classe mère.
- Si la sous classe ne définit pas son propre constructeur, elle va hériter du constructeur de la classe mère.
- Python supporte aussi l'héritage multiple. Une classe peut être étendue de plusieurs classes mères.

L'HÉRITAGE EN PYTHON

- Soit l'exemple suivant:
  - Animal: La class simule un animal.
  - Duck: La sous class de l' Animal.
  - Cat: La sous class de l' Animal.
  - Mouse: La sous class de l' Animal.



LES MÉTHODES ET LE CONSTRUCTEUR

- Le constructeur des sous classes appelle toujours le constructeur de la classe mère pour initialiser les valeurs des attributs dans la classe mère, et puis il va attribuer des valeurs à ses attributs.
- Soit la classe Animal

```
class Animal :

    # Constructeur
    def __init__(self, name):

        # La classe Animal a un attribut : 'name'.
        self.name= name

    # La méthode (method):
    def affiche(self):
        print ("Je suis " + self.name)

    # La méthode (method):
    def seDeplacer(self):
        print ("Je bouge ...")
```

LES MÉTHODES ET LE CONSTRUCTEUR

- Pour dériver d'une autre classe la syntaxe est :

  class Fille(Mere):

- Cat est la classe qui hérite de l' Animal, elle possède également ses attributs.
- L'appel au constructeur de la classe mère se fait par **super()**

```
from animal import Animal

# La classe Cat est étendue de la classe Animal.
class Cat (Animal):

    def __init__(self, name, age, height):
        # Appel au constructeur de la classe mère (Animal)
        # pour attribuer la valeur au attribut 'name' de la classe mère.
        super().__init__(name)
        self.age = age
        self.height = height

    # Redéfinir la méthode ayant le même nom de la classe mère.
    def affiche(self):

        print ("I'm " + self.name)
        print (" age " + str(self.age))
        print (" height " + str(self.height))
```

TEST DE LA CLASSE FILLE

```python
from Cat import Cat

tom = Cat("Tom", 3, 20)

print ("Appel méthode deplacer()")

tom.seDeplacer()

print("\nAppel Méthode affiche")

tom.affiche()
```

```
Appel méthode deplacer()
Je bouge ...

Appel Méthode affiche
I'm Tom
Age 3
Height 20
```

TEST DE LA CLASSE FILLE

```
from Animal import Animal

# La classe Cat est étendue de la classe Animal.
class Cat (Animal):

    def __init__(self, name, age, height):
        # Appel au constructeur de la classe mère (Animal)
        # pour attribuer la valeur au attribut 'name' de la classe mère.
        super().__init__(name)
        self.age = age
        self.height = height

    # Redéfinir la méthode ayant le même nom de la classe mère.
    def affiche(self):

        print ("I'm " + self.name)
        print (" age " + str(self.age))
        print (" height " + str(self.height))
```

MÉTHODES : UTILISER - REDÉFINIR - ENRICHIR - DÉFINIR

1. Je garde le comportement (méthode) offert par la classe mère : seDeplacer()

2. Je redéfinie le comportement (méthode) offert par la classe mère complètement :afficher()

3. Je l'enrichie avec super()

4. Je rajoute toutes les méthodes désirées sauter()

Exercice

Créer une classe PC qui contient :

Attributs :

- marque (str)
- processeur (str)
- ram (int, exprimée en Go)

Méthodes :

- afficher_info() → affiche les informations du PC (marque, processeur, ram)

Classe fille : Desktop (hérite de PC)

Cette classe représente un ordinateur fixe.

Attributs supplémentaires :

- taille_tour (str) — exemple : "grande", "moyenne", "petite"

Méthode supplémentaire :

- afficher_info() → affiche "PC de bureau avec une tour de taille <taille_tour>."

Classe fille : Laptop (hérite de PC)

Cette classe représente un ordinateur portable.

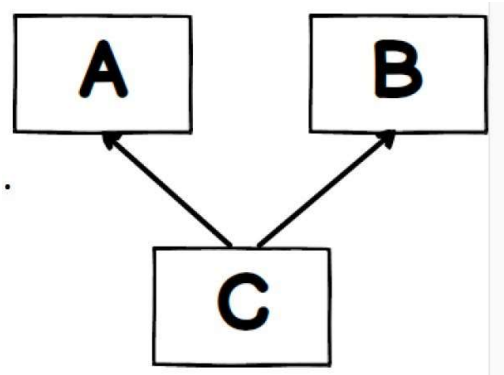
Attributs supplémentaires :

- poids (float, en kg)
- autonomie (int, en heures)

Méthodes supplémentaires :

- afficher_info() → affiche "ordinateur portable pesant <poids> kg avec <autonomie> h d'autonomie."

HÉRITAGE MULTIPLE



- Python prend en charge l'héritage multiple.
- Nous spécifions toutes les classes parentes sous forme de liste séparée par des virgules entre parenthèses.

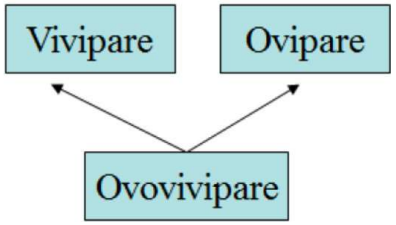
```
class A(object):
    def __init__(self):
        self.str1 = "Hello"
        print("A")
class B(object):
    def __init__(self):
        self.str2 = "World"
        print("B")
```

```
class C(A, B):
    def __init__(self):
        # Appel le constructeur de la classe A et B
        A.__init__(self)
        B.__init__(self)
        print("C")
    def afficherMsg(self):
        print(self.str1, self.str2)
c = C()
c.afficherMsg()
```

HÉRITAGE MULTIPLE

```python
class Ovipare :
        def __init__(self,nom0):
            self.nom=nom0
            print("constructeur ovipare")
        def affiche(self):
            print("Appel méthode affiche ovipare :"+self.nom)

class Vivipare :
        def __init__(self,t):
            self.type=" de type "+t
            print("constructeur vivipare")
        def affiche(self):
            print("Appel méthode affiche vivipare :"+self.type)

#héritage multiple
class Ovovivipare(Vivipare,Ovipare):
    def __init__(self,aName,aType):
        Ovipare.__init__(self,aName)
        Vivipare.__init__(self,aType)
        print("constructeur Ovovivipare "+aName)


ov=Ovipare("Poule")
ov.affiche()
vi=Vivipare("Snake")
vi.affiche()
ovovi=Ovovivipare("Python","Snake")
ovovi.affiche()
```

- Zoologie




```
constructeur ovipare
Appel méthode affiche ovipare :Poule
constructeur vivipare
Appel méthode affiche vivipare : de type Snake
constructeur ovipare
constructeur vivipare
constructeur Ovovivipare Python
Appel méthode affiche vivipare : de type Snake
```

HÉRITAGE MULTIPLE

```python
class Ovipare :
        def __init__(self,nomO):
            self.nom=nomO
            print("constructeur ovipare")
        def affiche(self):
            print("Appel méthode affiche ovipare :"+self.nom)

class Vivipare :
        def __init__(self,t):
            self.type=" de type "+t
            print("constructeur vivipare")
        def affiche(self):
            print("Appel méthode affiche vivipare :"+self.type)

#héritage multiple
class Ovovivipare(Vivipare,Ovipare):
    def __init__(self,aName,aType):
        Ovipare.__init__(self,aName)
        Vivipare.__init__(self,aType)
        print("constructeur Ovovivipare "+aName)


ov=Ovipare("Poule")
ov.affiche()
vi=Vivipare("Snake")
vi.affiche()
ovovi=Ovovivipare("Python","Snake")
ovovi.affiche()
```

- Les deux classes mère offrent la méthode affiche
- La classe fille ne redéfinit pas la méthode affiche
- C'est la méthode affiche de la 1ère classe mère qui est appelée

```
constructeur ovipare
Appel méthode affiche ovipare :Poule
constructeur vivipare
Appel méthode affiche vivipare : de type Snake
constructeur ovipare
constructeur vivipare
constructeur Ovovivipare Python
Appel méthode affiche vivipare : de type Snake
```

HÉRITAGE MULTIPLE

```
9   class Ovipare :
10          def __init__(self,nom0):
11              self.nom=nom0
12              print("constructeur ovipare")
13          def affiche(self):
14              print("Appel méthode affiche ovipare :"+self.nom)
15
16  class Vivipare :
17          def __init__(self,t):
18              self.type=" de type "+t
19              print("constructeur vivipare")
20          def affiche(self):
21              print("Appel méthode affiche vivipare :"+self.type)
22
23  #héritage multiple
24  class Ovovivipare(Vivipare,Ovipare):
25      def __init__(self,aName,aType):
26          Ovipare.__init__(self,aName)
27          Vivipare.__init__(self,aType)
28          print("constructeur Ovovivipare "+aName)
29
30
31  ov=Ovipare("Poule")
32  ov.affiche()
33  vi=Vivipare("Snake")
34  vi.affiche()
35  ovovi=Ovovivipare("Python","Snake")
36  ovovi.affiche()
```

```
constructeur ovipare
Appel méthode affiche ovipare :Poule
constructeur vivipare
Appel méthode affiche vivipare : de type Snake
constructeur ovipare
constructeur vivipare
constructeur Ovovivipare Python
Appel méthode affiche vivipare : de type Snake
```

HÉRITAGE MULTIPLE

```python
class Ovipare :
        def __init__(self,nomO):
            self.nom=nomO
            print("constructeur ovipare")
        def affiche(self):
            print("Appel méthode affiche ovipare :"+self.nom)

class Vivipare :
        def __init__(self,t):
            self.type=" de type "+t
            print("constructeur vivipare")
        def affiche(self):
            print("Appel méthode affiche vivipare :"+self.type)

#héritage multiple
class Ovovivipare(Ovipare, Vivipare):
    def __init__(self,aName,aType):
        Ovipare.__init__(self,aName)
        Vivipare.__init__(self,aType)
        print("constructeur Ovovivipare "+aName)

ov=Ovipare("Poule")
ov.affiche()
vi=Vivipare("Snake")
vi.affiche()
ovovi=Ovovivipare("Python","Snake")
ovovi.affiche()
```

```
constructeur ovipare
Appel méthode affiche ovipare :Poule
constructeur vivipare
Appel méthode affiche vivipare : de type Snake
constructeur ovipare
constructeur vivipare
constructeur Ovovivipare Python
Appel méthode affiche ovipare :Python
```

HÉRITAGE MULTIPLE : propriétés

En Python, **l'ordre d'héritage** dans la déclaration de la classe (class C(A, B)) détermine la **priorité** des classes mères.

•Python suit un ordre appelé **MRO (Method Resolution Order)**.

•Il cherche la méthode dans la classe **C**, puis dans **A**, puis dans **B**, puis enfin dans **object** (la super-classe de tout en Python).

Tu peux le vérifier avec :

print(C.mro())

```
class A:
    def parler(self):
        print("Je suis A")

class B:
    def parler(self):
        print("Je suis B")

class C(A, B):
```

```
print(C.mro())
```

```
[<class '__main__.C'>, <class '__main__.A'>, <class '__main__.B'>, <class 'object'>]
```

HÉRITAGE MULTIPLE : propriétés

- Si les classes mères disposent d'une méthode avec le même nom

➔ l'heritage pointe sur la méthode appartement à la première classe mère évoquée dans l'héritage

```
class A:
    def parler(self):
        print("Je suis A")

class B:
    def parler(self):
        print("Je suis B")

class C(A, B):
```

```
c = C()
c.parler()
```

```
Je suis A
```

HÉRITAGE MULTIPLE : propriétés

il y a deux attributs distincts prix :
il n'existe qu'un **seul attribut d'instance self.prix**, et sa dernière affectation détermine sa valeur.

Quand on crée un objet C() :
1.L'appel à A.__init__(self) crée **l'attribut d'instance self.prix et le met à 10**.
2.Puis, l'appel à B.__init__(self) **remplace** cet attribut par une nouvelle valeur 20.

```
class A:
    def __init__(self):
        self.prix = 10

class B:
    def __init__(self):
        self.prix = 20

class C(A, B):
    def __init__(self):
        A.__init__(self)
        B.__init__(self)
```

```
c = C()
print(c.prix)  # Résultat : 20
```

HÉRITAGE MULTIPLE : propriétés

*** Possibilité d'appeler les methods ayant les mêmes noms en se référant à la classe mère**

```python
class MachineMecanique:
    def __init__(self):
        self.type_mecanique = "mécanique simple"

    def demarrer(self):
        print("La machine mécanique démarre.")
```

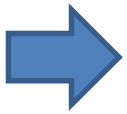
```python
class MachineElectrique:
    def __init__(self):
        self.type_electrique = "basse tension"

    def demarrer(self):
        print("La machine électrique démarre.")
```

```python
class Appareil(MachineMecanique, MachineElectrique):
    def __init__(self):
        # Appel explicite des constructeurs des deux classes mère
        MachineMecanique.__init__(self)
        MachineElectrique.__init__(self)

        self.nom = "Appareil hybride"

    def demarrer(self):
        print(f"{self.nom} démarre :")
        # Appels explicites des méthodes des classes mères
        MachineMecanique.demarrer(self)
        MachineElectrique.demarrer(self)
        print("Appareil prêt à fonctionner !")
```

HÉRITAGE MULTIPLE : propriétés

*** Possibilité d'appeler les methods ayant les mêmes noms en se référant à la classe mère**

```
app = Appareil()
print(f"Type mécanique : {app.type_mecanique}")
print(f"Type électrique : {app.type_electrique}")
app.demarrer()
```



```
Type mécanique : mécanique simple
Type électrique : basse tension
Appareil hybride démarre :
La machine mécanique démarre.
La machine électrique démarre.
Appareil prêt à fonctionner !
```

HÉRITAGE MULTIPLE : Héritage de 3 classes mères

```python
# Première classe mère
class MachineMecanique:
    def __init__(self):
        self.type_mecanique = "mécanique simple"

    def demarrer(self):
        print("La machine mécanique démarre avec un moteur à engrenages.")

# Deuxième classe mère
class Volume:
    def __init__(self):
        self.volume = 10  # en litres

    def afficher_volume(self):
        print(f"Volume : {self.volume} L")

# Troisième classe mère
class MachineElectrique:
    def __init__(self):
        self.type_electrique = "basse tension"

    def demarrer(self):
        print("La machine électrique démarre avec un courant de 220V.")
```

```python
# Classe fille qui hérite des trois
class Appareil(MachineMecanique, Volume, MachineElectrique):
    def __init__(self):
        # Appel explicite à tous les constructeurs parents
        MachineMecanique.__init__(self)
        Volume.__init__(self)
        MachineElectrique.__init__(self)

        self.nom = "Appareil multifonction"

    # Redéfinition (override) de la méthode 'demarrer'
    def demarrer(self):
        print(f"{self.nom} se prépare à démarrer...")
        # On appelle explicitement les méthodes des parents :
        MachineMecanique.demarrer(self)
        MachineElectrique.demarrer(self)
        print("Appareil prêt à fonctionner !")
```

MERCI